Week 9
Game Engine Development Unity Animation

Hooman Salamat

Objective & Concept

Objective

- To learn how to implement animation in Unity.
- To bring motion and the illusion of life to characters and environments by creating and configuring animations in the Unity Editor.

Concepts

- The goal of this learning experience is to introduce you to the most common ways that animation is used in Unity for various interactive experiences. You'll be able to use what you learn here to apply animation to your own projects.
- There are two major types of animation that you'll work with in Unity: animations that you import from another program, and animations that you create within the Editor itself. In this project you'll learn about both, and how they can be applied to your own personal projects.

What does an animator do in Unity?

- Unity animators are responsible for animating objects for the interactive experience, ranging from characters and set pieces to cutscenes and even interfaces. Practically anything in Unity can be animated!
- Typically, highly complicated objects or in-depth animations are created in external **Digital Content Creation** (DCC) applications, like Autodesk Maya or Blender.
- These DCCs have robust suites of animation tools to help simplify the creation process for the animator.
- These generally include tools for creating **rigs** (the underlying skeletal-like structure for animating characters), dynamics such as hair and fur, saved poses, and more.
- It's for this reason that in-editor animation is saved for more simple objects, or when the animation is tied to a specific scene; for example, a platform moving between two points in a game level. It's extremely common for both externally and internally created animations to be used in the same project.

Tools

In addition to Unity, animators often other software to produce animations, which may include:

- Autodesk Maya: Long considered the industry standard for animation creation in film, TV, animated features, and games. Maya is a robust digital art creation tool that can be used for many purposes outside of animation as well, including model generation and visual effects.
- Autodesk 3ds Max: Similar to Maya, 3ds Max is a multifunctional digital art creation tool and was historically the software of choice for the games industry.
- Blender: Completely free and open source, Blender has long been a popular choice for creators in the indie community, but is beginning to gain serious traction in professional markets as well. Like Max and Maya, Blender is a multipurpose tool that features a robust collection of animation tools.

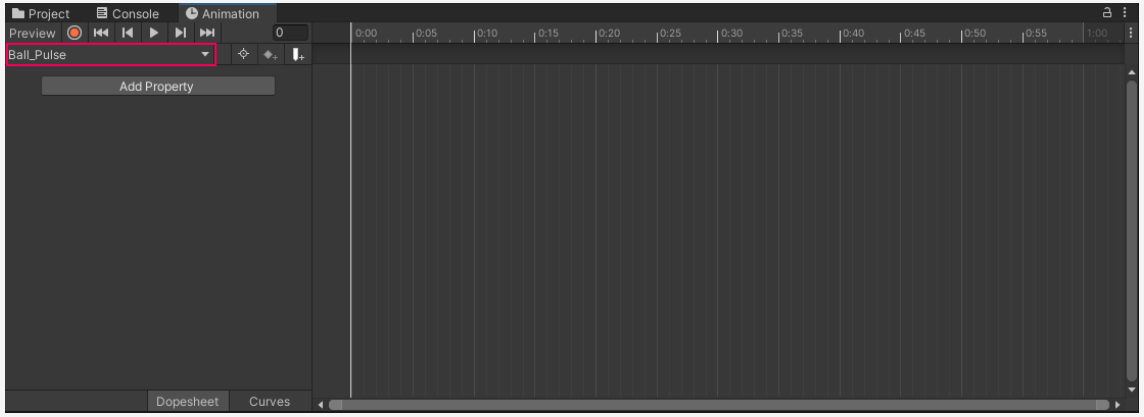
Why is animation important?

- Animation is sometimes referred to as the illusion of life. There's an energy to living things, and this is often translated into some form of controlled motion.
- Birds fly, people dance, cheetahs run—and recreating all of these actions digitally can be accomplished through the use of animation.
- Animation isn't necessarily just movement, as movement can be accomplished through other methods, like using physics.
- A rock falling to the ground doesn't need animation to get from point A to point B—a physics system can mathematically calculate gravitational force to accomplish that goal.
- Instead, it's movement with intention where animation comes into play: physics can't simulate the process of a person walking.

Animation Clip

- In Unity, individual animations are stored in an asset called an **Animation Clip**.
- They are associated with the object selected at the time of the clip's creation, and store all data that goes into creating a specific animation.
- Animation clips can be any length of time, and a single GameObject can have multiple Animation Clips associated with it.

Animation Editor

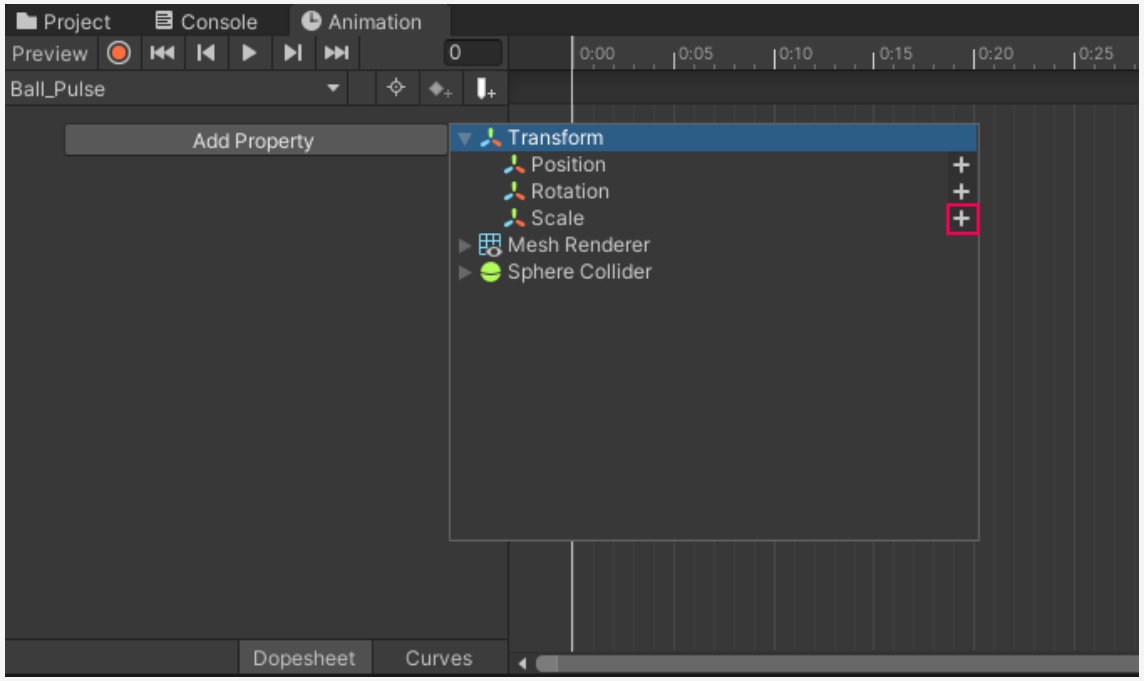


- Open the Animation editor by navigating to the Window drop down and going to **Animation** > **Animation.**
- When you save the new Animation Clip, the appearance of the Animation window changes.
- The left side of the window now features an **Add Property** button where you can add and view the properties you wish to animate.
- Directly above this button is the name of your current Animation Clip in a dropdown menu.
- If you choose to have multiple animations on the same GameObject, you can select the one you wish to view from this menu.

Animation keyframes

- Just as an animation is contained within an Animation Clip, the specific poses within an animation are defined by its **keyframes**.
- In traditional hand-drawn animation, keyframes are the poses that lead animators draw, and are important (or key) drawings that define the animation sequence.
- Once the lead animators finish these key poses, other animators draw the frames between them to complete the animation.
- These frames are referred to as in-between frames, also known as **tweens**.

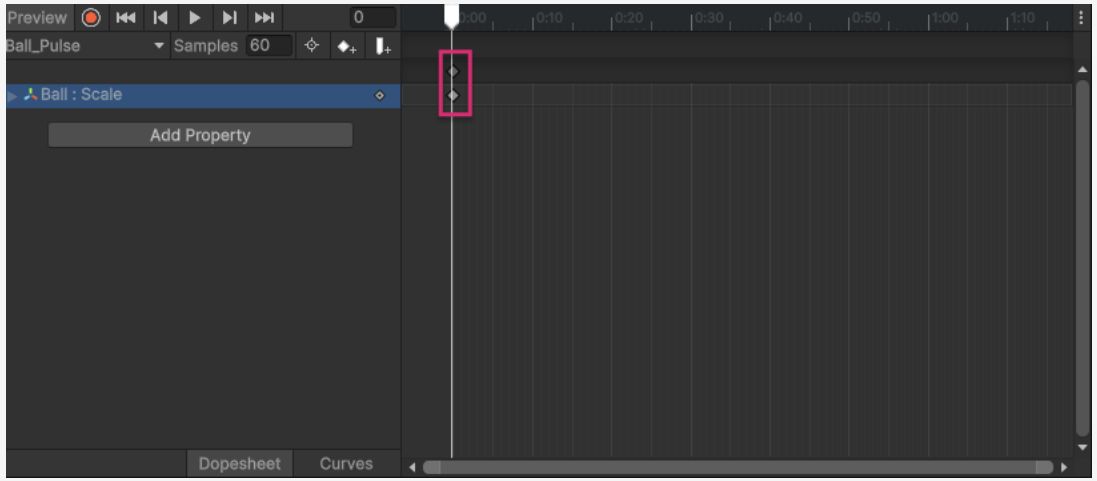
Tweening



In digital animation, tweening is generally handled by whatever tool the animator is working with, meaning they only need to worry about setting up keyframes.

Digital keyframes record all the data related to the GameObject being animated at a defined time stamp.

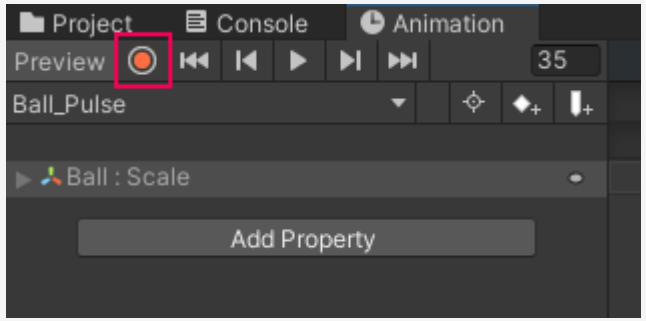
Keyframes



- When a new property is added to an animation, the Animation editor records the relevant data (in this case, the ball's current scale) about the selected object and generates initial keyframes for it.
- Two diamond icons will appear at the start of the timeline.
- These icons represent keyframes.

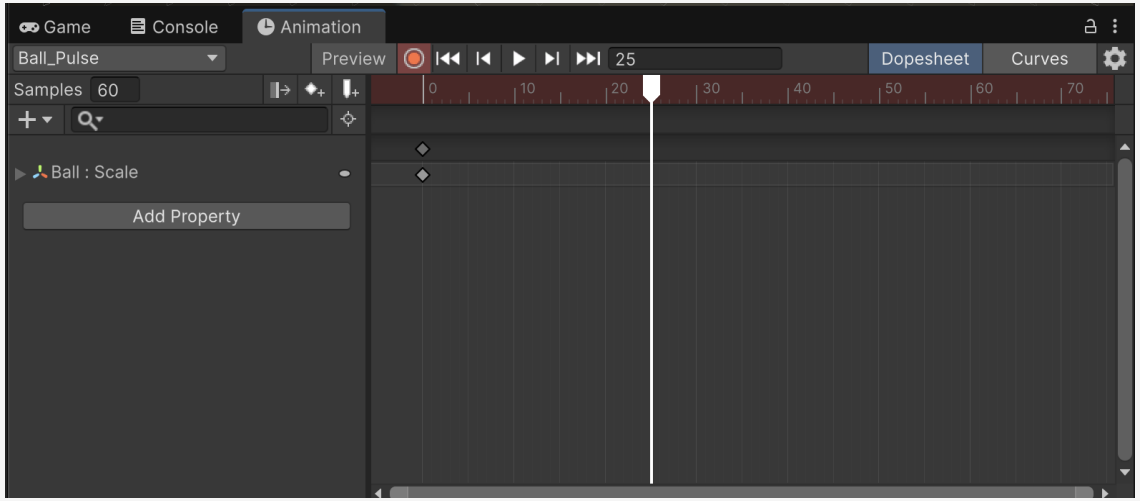
Record button

- Once the **Record** button is pressed, both it and the timeline will be highlighted in red.
- So long as the **Record** button is active, Unity will automatically add keyframes for any property that is changed in the Inspector.



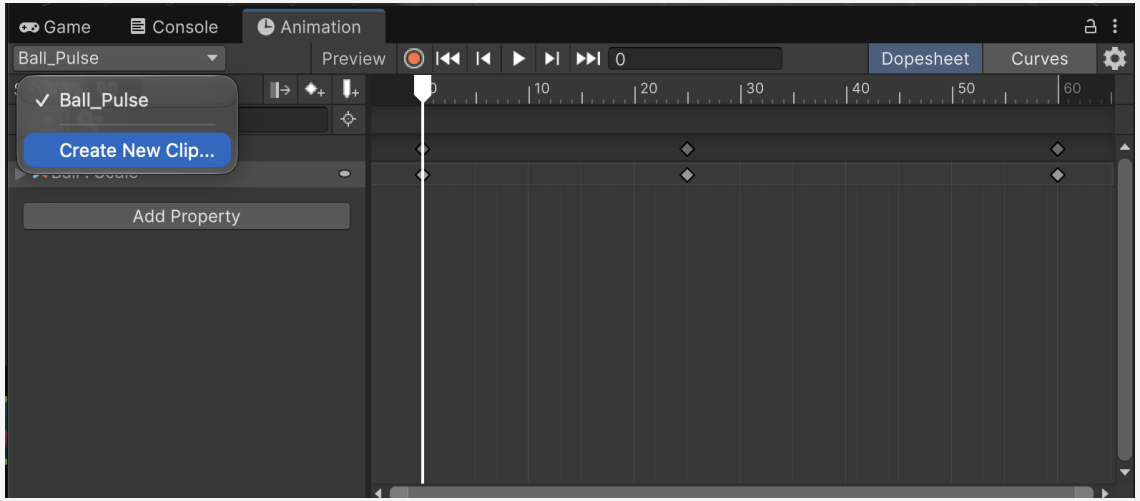
Playhead

- The white vertical line is called the **Playhead**.
- The Playhead indicates which frame is currently active and where new keyframes will be created.

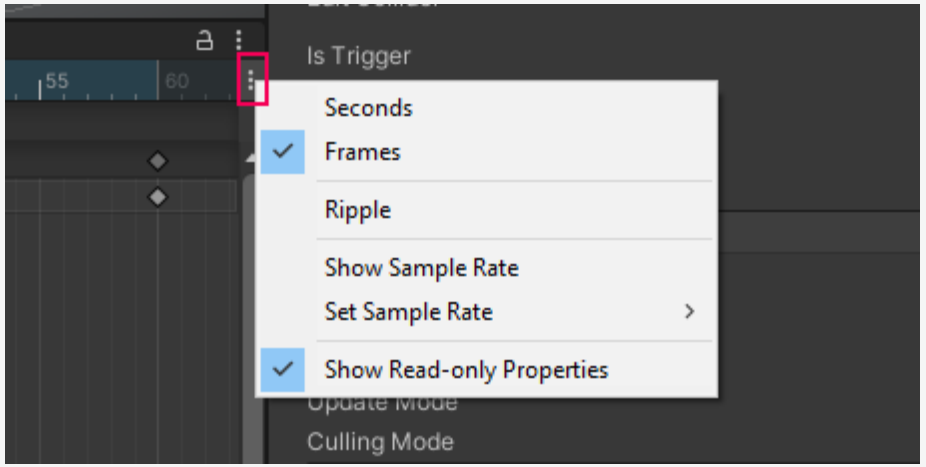


Animation Clips

- A single object can have an infinite number of Animation Clips.
- Example: There are three major points to create keyframes for: the ball falling, the ball striking the ground, and the ball bouncing back up.

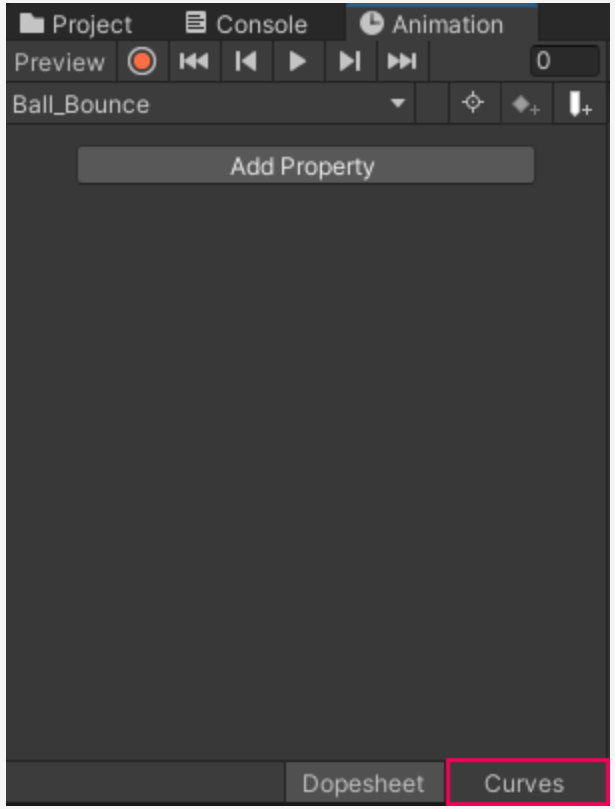


Sample Rate



- By default, Unity animations have a frame rate (FPS) of 60. This means that for every second of animation, 60 frames are rendered on the screen. This produces very smooth animation; however, the more common frame rate for animation is 24 FPS. This is also the frame rate for most movies. The frame rate that you choose to animate in ultimately comes down to personal preference, but make sure you use the same rate consistently across all animations in your project. This is especially important to keep in mind if you intend to use imported animations from a DCC or the Unity Asset Store, which are often at 24 FPS.
- Click on the same three vertical ellipsis button from the previous step, and go to **Set Sample Rate > 24.**

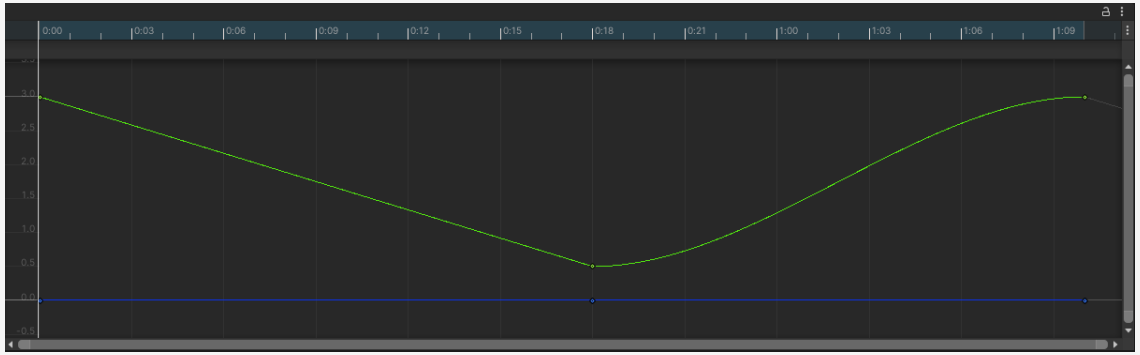
Curve Editor



- The curve shown in the editor is a visual representation of the tweens occurring between the set keyframes, specifically their transition rate.
- In cases like the bounce animation, an animator must modify the tweens themselves, and in Unity this is done through the **Curves editor**.
- Until this point you have been working in a section of the Animation editor called the **Dopesheet**, which is where all keyframes are set.

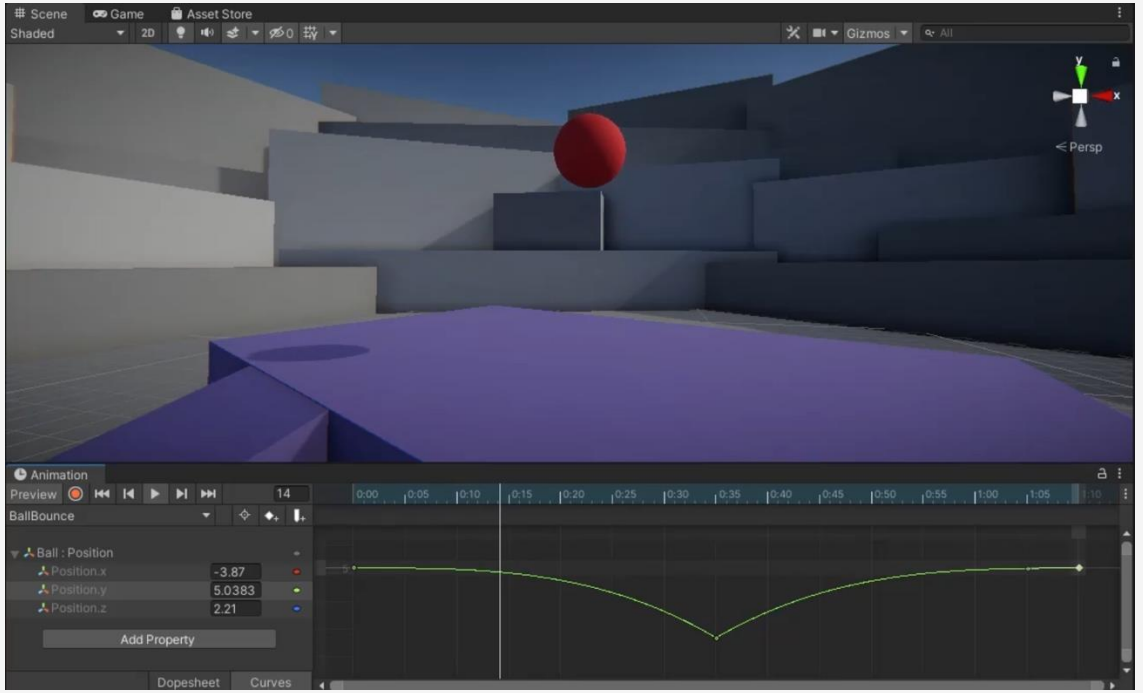
Easing

- The curve shown in the editor is a visual representation of the tweens occurring between the set keyframes, specifically their transition rate. If you preview the animation, you can see this relationship in action. For the first few frames of the animation, the ball slowly descends from its full height, picks up speed in the middle of the drop, then slows down for the landing. It repeats this process in reverse to return to the starting point. This type of transition is referred to as **easing**.
- Let's explore how the animation changes when no easing is applied.
  - Right-click the first keyframe of the animation in the Curve editor and from the list select **Right Tangent** > **Linear**.
  - Right-click the middle keyframe, and from the list select **Left Tangent** > **Linear**.



Adjust the easing of the ball

- Right-click the middle keyframe and from the list, go to **Both Tangents** > **Free**. This will keep the tangents in the same position, but give you access to the tangent handles.
- Adjust the tangent handles of the middle keyframe to form a sharper V shape.



Create squash keyframes

- When an object strikes a surface with sufficient force, deformation of either the object or the surface will occur. The deformed object will then return to its original form (or as close as possible) once the energy of the strike has dissipated.
- In animation, this process is encapsulated in a technique called squash and stretch.
- In traditional animation this process is often overemphasized to give impacts a bigger presence.

Create squash keyframes in Unity

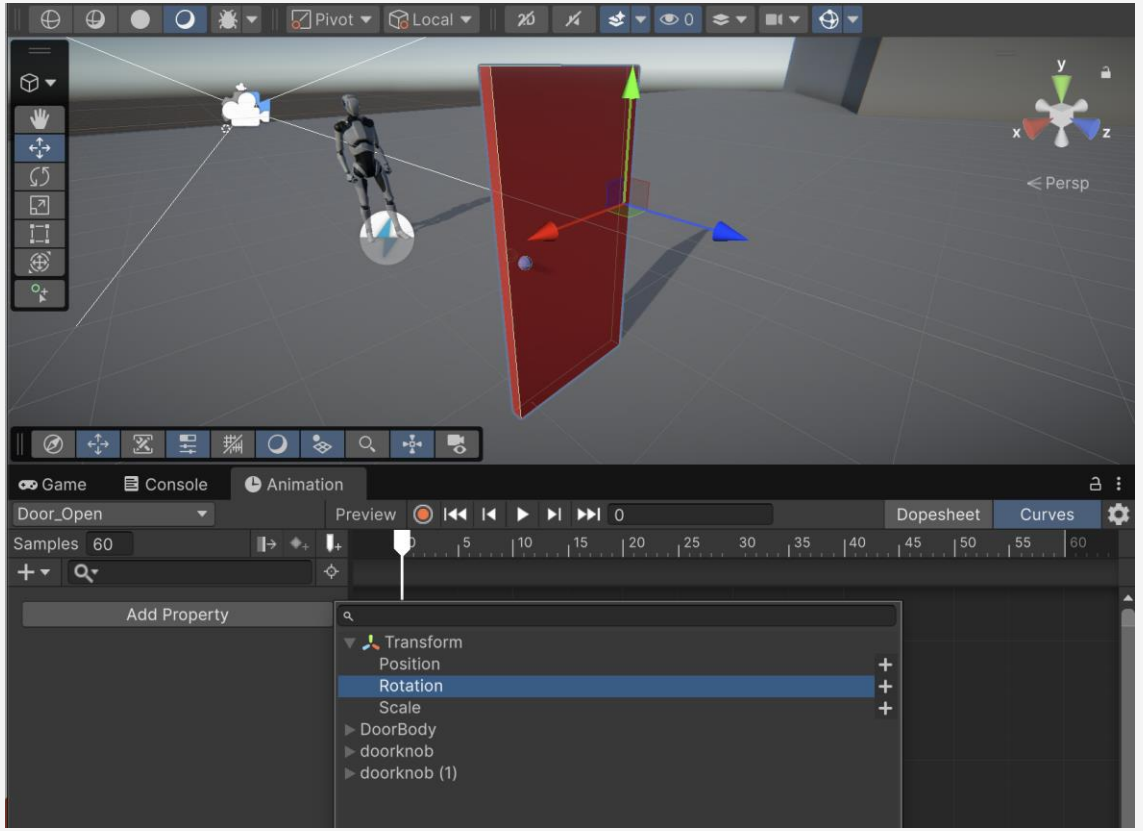
- Return to the Dopesheet by clicking on the Dopesheet button on the bottom of the Animation editor.
- Set the Playhead to the first keyframe at 0:00 and turn on keyframe recording.
- Click Add Property and go to **Transform** > **Scale**, and create a keyframe in the 1:10 mark.
- Advance the Playhead to the second keyframe, which is the moment the sphere strikes the ground.
- Press R on your keyboard to enable the Scale Tool. Scale the sphere down to 0.8 in Y.
- Press W on your keyboard to enable the Move Tool. Drag the sphere down so it appears to be resting on the surface of the ground again.

Create stretch keyframes

- With the squash impact of the ball complete, you can move on to the stretch. For a rubber ball, this occurs right after the impact with the ground, when the energy from the fall launches the ball back into the air.
- Advance the Playhead to the frame immediately after the impact frame.
- In the Inspector, set the Y scale value to 1.2.
- Preview the animation by pressing play in the Animation editor. The sphere will now squash and stretch as it hits and bounces back up into the air.
- Adjust the squash and stretch keyframes to your personal preferences.
- Click the record button to exit record mode once you're finished editing.

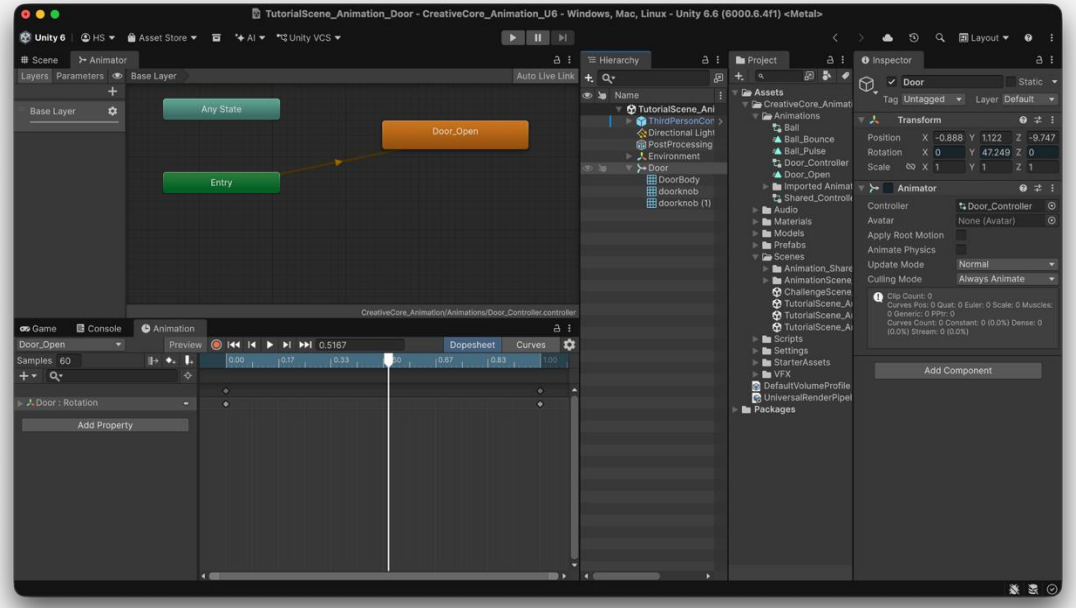
Door Animation

- Before you begin animating, it's important to check where the door's **pivot point** is.
- A pivot point is the point on an object where movement occurs. In previous animation exercises, the location of the pivot point didn't matter, but doors are different because they move in a very specific way.
- It's important to replicate the proper kind of movement so the animation doesn't look odd! For doors, the pivot point is where the door would attach to its frame on a wall.



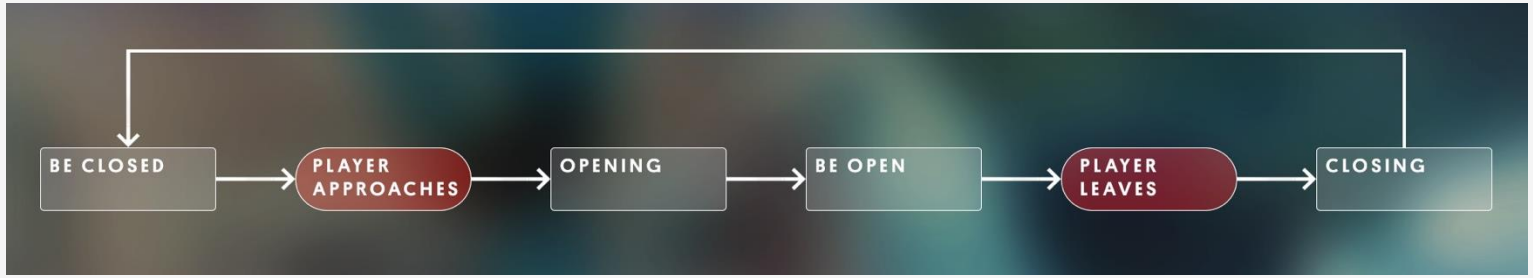
Animator Controller

- Animator components are automatically created whenever the first Animation Clip is created for a selected GameObject.
- The Animator is responsible for assigning animation; however, it doesn't control the actual Animation Clips.
- This task falls to the Animator Controller, which is also automatically created with the first Animation Clip.



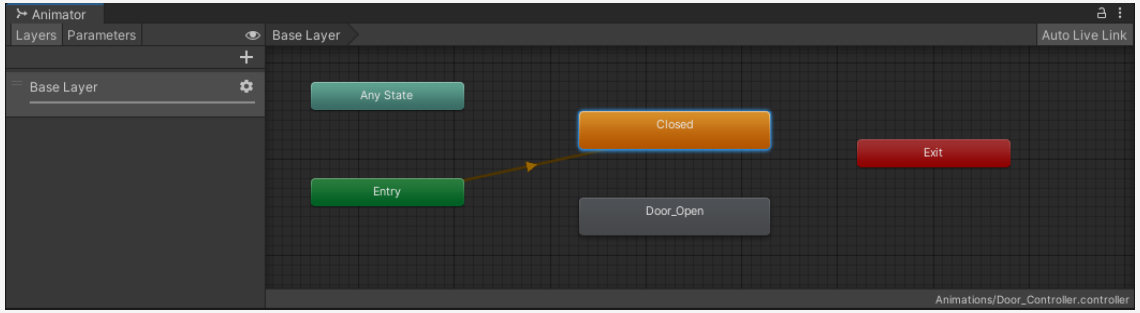
State Machine

- Animation Controllers run on a special type of system called a **State Machine**.
- A State Machine keeps track of all of the possible actions an object can perform, and selects the appropriate one based on what the current situation (or state) calls for.
- The logic that communicates the current state is controlled externally, usually via a script, but the process of moving from one action to another is handled directly by a State Machine and is called a **Transition.**



Create the default state

- Currently, the Door_Open animation plays automatically as soon as the scene is run.
- This is because it's currently the only animation associated with the door, and was automatically added as the Controller's default state.
- However, the starting state of the door should actually be the Closed state, which doesn't have any associated animation.
- To account for this, add a Closed node to the Controller and set it as the default state.



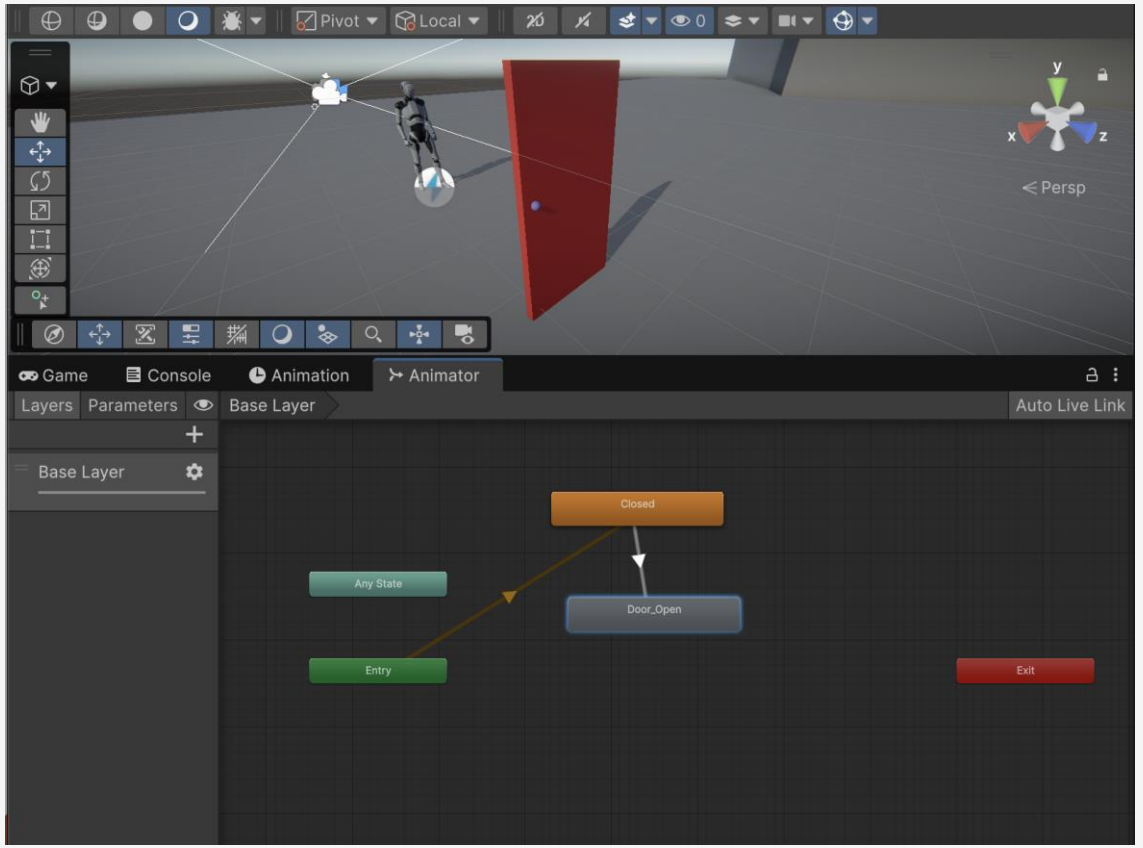
1. In the gridded Animator work area, right-click and select **Create State** > **Empty.**

2. Select the new state node, and in the Inspector, change the name from New State 0 to Closed.

3. Right-click the Closed state and select Set as Layer Default State.

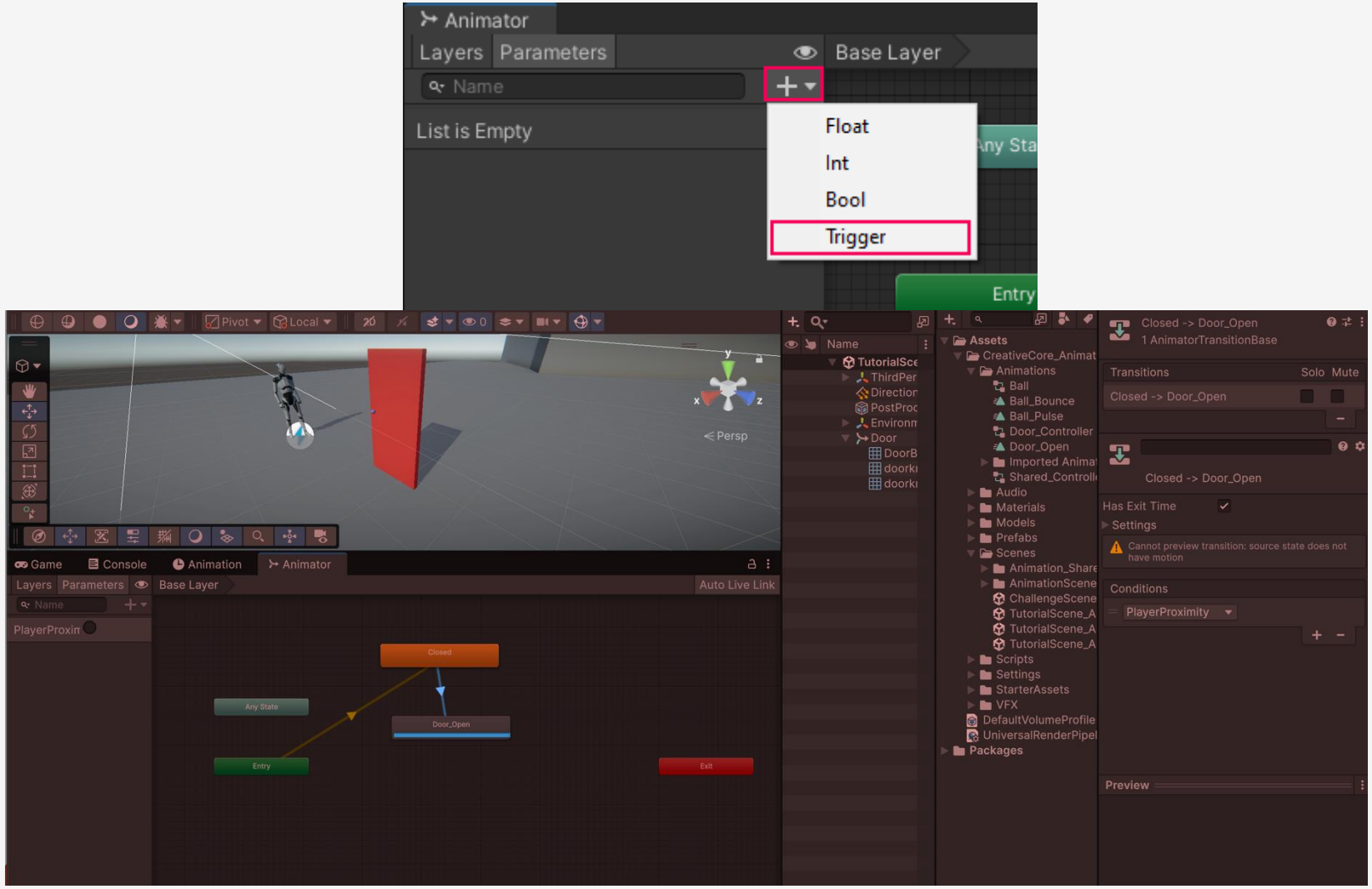
Open the door

- The Animation Controller will run through the Closed node once. When finished, it will immediately transition to the Door_Open animation, then sit idle at the end of the animation.
- In order to prevent one node immediately transitioning into another, some kind of logic gate must be placed on the transition.
- This is what the script you'll use will check for in the final version. This will be accomplished through the use of **Parameters**.



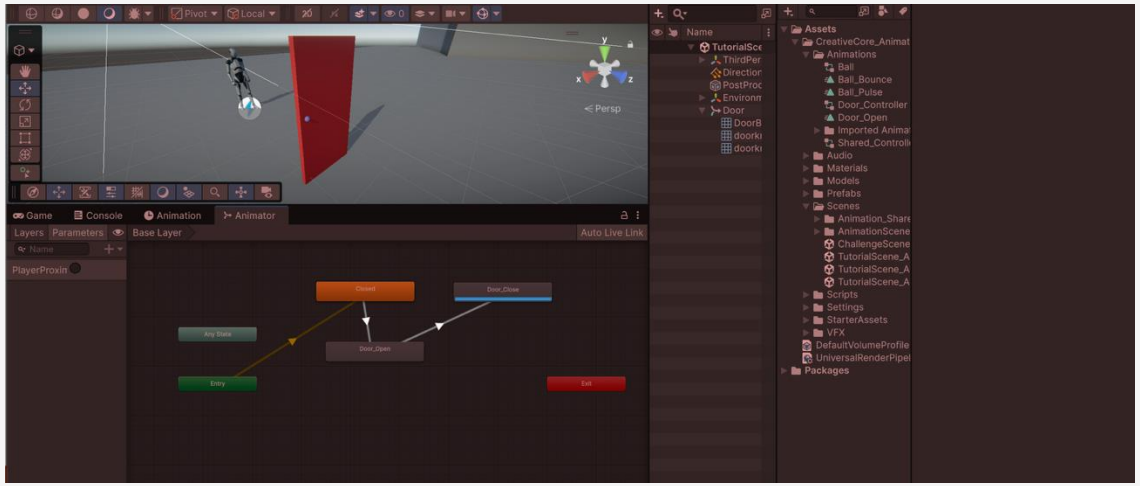
Parameters

- Parameters are variables that are created directly within the Animator, and can be found on the Parameters tab on the left side of the Animator editor.
- Select the Parameters tab to access the currently empty Parameter list.
- At the top of the Parameters list, click the **+** and select Trigger.
- A Trigger Parameter will immediately transition as soon as it's called via script.
- Name the Trigger PlayerProximity, taking care to spell and case it exactly as written here. It will be referenced in the script later.
- Select the Transition between Closed and Door_Open by clicking on the white arrow. In the Inspector, locate the Conditions section.
- Select the **+** at the bottom of the Conditions list. Since it's the only parameter, PlayerProximity will auto populate in the list.
- Play the scene.
- Select the **radio** button next to PlayerProximity in the parameters list to trigger it



Close the door

- Select the Door_Open node in the Animator and press **Ctrl/Cmd+D** on the keyboard to duplicate it.
- In the Inspector, rename the new node to Door_Close
- Still in the Inspector, locate the Speed parameter and set it to -1. This will reverse the animation.
- Right-click the Door_Open node and select Make Transition.
- Select the Door_Close node to connect the Transition between the two nodes.
- Select the new Transition and in the Inspector, click the **+** button in the Conditions list. PlayerProximity will auto populate.



Reset the State Machine

- The progression through the various door states is now complete; however, as it is right now, the door can only be interacted with once.
- In order to reuse the door infinitely, the State Machine must be reset back to its original state once the door close animation is complete.
- You'll be able to accomplish this by making use of the **Exit node**.
- When transitioned to, the Exit node will automatically transition to the Entry node, which pushes back to the default state. This will allow you to restart the Door's State Machine infinitely.

1. Right-click the Door_Close node and select Make Transition.

2. Click the Exit node to connect the transition between the two nodes.

3. Playtest the scene.

- Now when you press PlayerProximity to close the door, the Door_Close animation will play and automatically transition to Exit, and will loop back to the Closed state!
- You may have noticed that sometimes there's a slight delay between the time that you press PlayerProximity and when the door opens.
- This is because, by default, a state node will complete its active play time (represented by the progress bar) before making a transition. This can be disabled, making the transition to the Door_Open state immediate.

4. Select the transition between the Closed and Door_Open nodes.

5. In the Inspector, disable Has Exit Time.

Create the proximity trigger

1. In the Hierarchy right-click and select Create Empty.

2. Rename the Empty GameObject to Door Trigger.

3. With Door Trigger selected, click the **Add Component** button in the Inspector and search for Box Collider and add it.

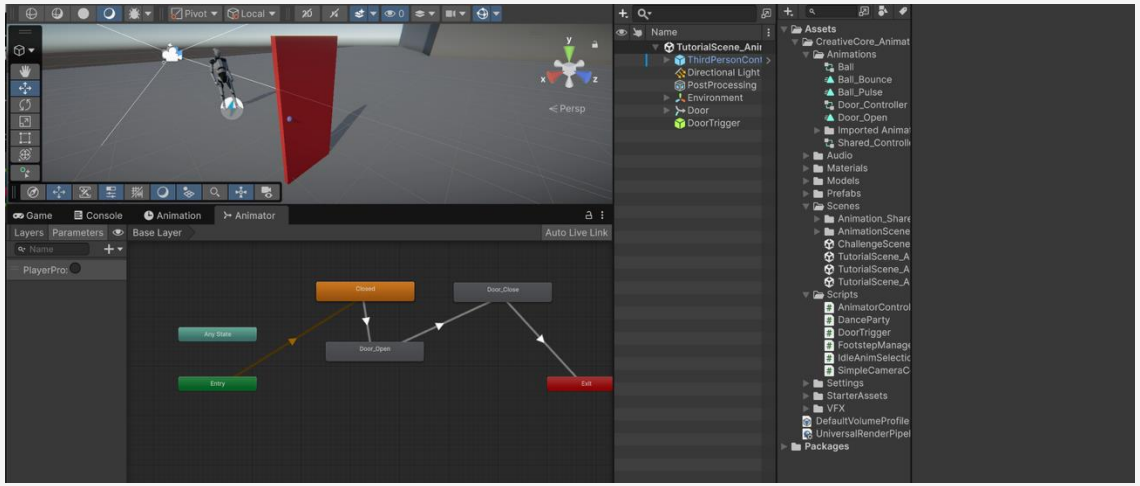
4. In the Box Collider component, enable **Is Trigger**. This will change the Box Collider to a Trigger.

5. Increase the Box Collider size to 3 in the Z axis.

6. Arrange the Door Trigger so that it overlaps the door mesh, with half of the trigger arranged on either side of the door. This will create a good-sized zone for the player controller to enter to trigger the door to open.

7. Locate the DoorTrigger script in the Project Script's folder, and add it to the Door Trigger GameObject.

8. Drag the Door GameObject into the Anim parameter of the DoorTrigger script. This will automatically assign the Door's animator to the parameter.



DoorTrigger Script

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;


public class DoorTrigger : MonoBehaviour
{
public Animator anim;


private void OnTriggerEnter(Collider other)
{
anim.SetTrigger("PlayerProximity");
}


private void OnTriggerExit(Collider other)
{
anim.SetTrigger("PlayerProximity");
}
}
```

Humanoid Animation System

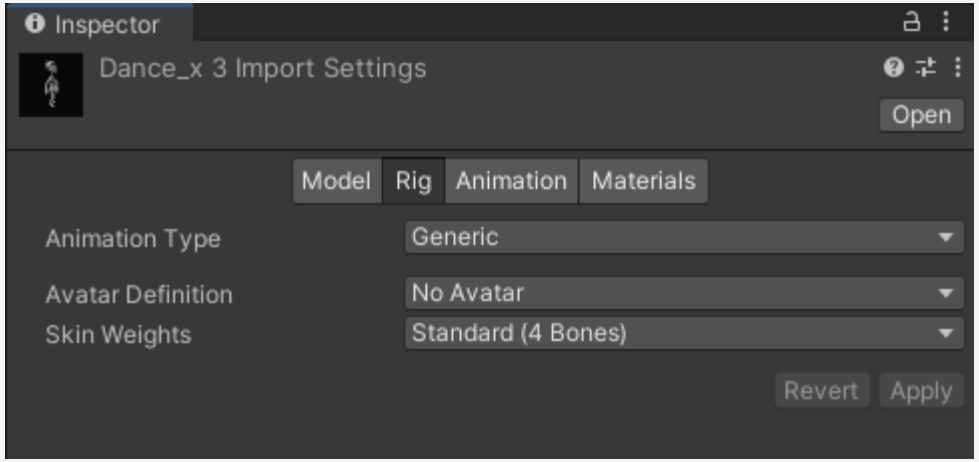
- Animating can be a very time intensive task. Often, animators have to decide where best to spend their time on a project, and may opt to use pre-built animations for background characters, or for animation sets that don't need to be unique. This allows them to work on what the user will see the most in the interactive experience.
- Not all character types can easily make use of animation built for another rig, but in Unity, any character that follows a basic humanoid shape (a head, two legs, two arms) can make use of the **Humanoid Animation System**.
- In this system, Unity translates existing animation from one humanoid character to another based on the joints in the rig. This means that even drastically different looking characters can share the same animation set!

Configure the animation type

- When preparing animation for import into Unity, an animator is able to create different files for each individual animation, or animate everything together in one file.
- When a single file containing multiple animations is imported into Unity, it can be split up in the import settings in the Clips section.

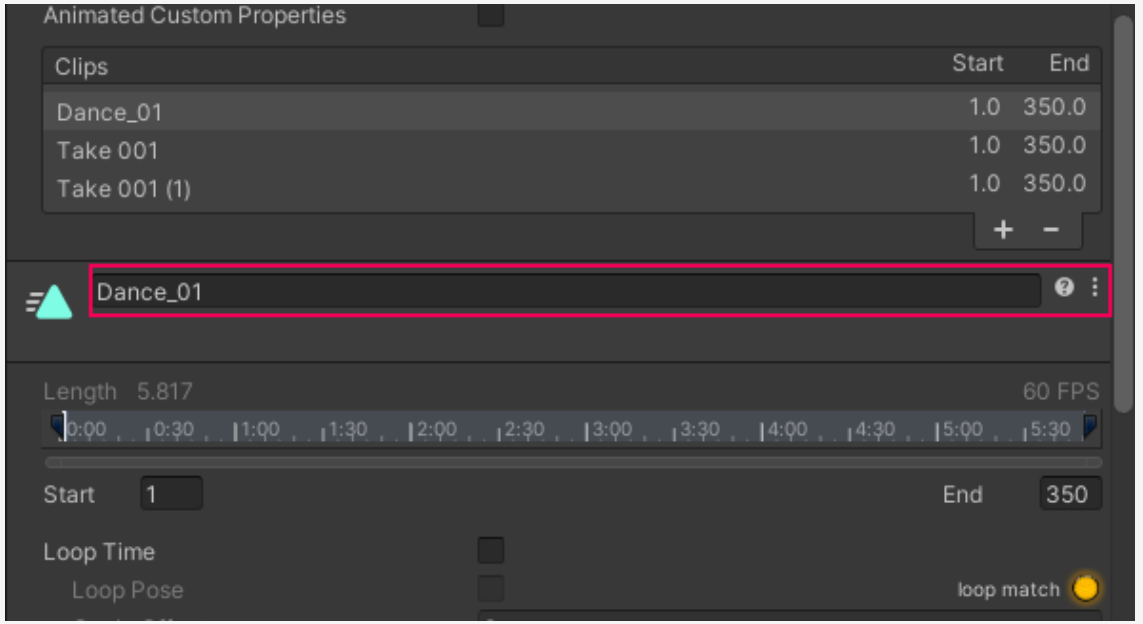
Rig tab

- By default, even rigs that are compatible with the Humanoid Animation system will have an **Animation Type** set to **Generic**.
- The Generic rig is useful when working with custom characters that have non-standard bone structures, as it prevents you from sharing the animations with other humanoid rigs.
- The **Humanoid** rig type, on the other hand, is ideal for humanoid characters with standard bone structures;
- it allows you to take advantage of the animation retargeting and sharing capabilities provided by Unity's Humanoid Animation System.



Configure the Animation Clip

- The imported animation set contains three distinct dance sequences in one Animation Clip. Currently, these animations can't be accessed individually, so let's fix that next. The top half of the Animation import settings contains the Clips section; this is where you can define the individual Animation Clips.
- In the clips section, click the **+** button twice to duplicate the existing Animation Clip.
- Select the first clip labeled Dance_x3 to select it in the clip editing section below Clips and rename the Dance_x3 clip to Dance_01.
- In the end frame parameter, type 150.
- Preview the animation and make sure that it is looping seamlessly, and adjust the end frame if needed.
- The animation is looping in the preview, but in the actual experience it's currently set to only animate once.
- Enable **Loop Time** so the animation will loop.



Configure the two remaining clips

- The process you just completed for the first clip must now be repeated for the remaining two clips.

  1. Select the second clip in the Clips list, currently named Take 001, and rename it to Dance_02.

  2. Set the Start frame to 151.

  3. Set the End frame to 260.

  4. Enable **Loop Time**.

  5. Enable the **Bake into Pose** option in Root Transform Rotation, Root Transform Position (Y), and Root Transform Position (XZ).

  6. Select the final clip in the Clips list and rename it to Dance_03.

  7. Set the Start frame to 261 and leave the End frame at 350.

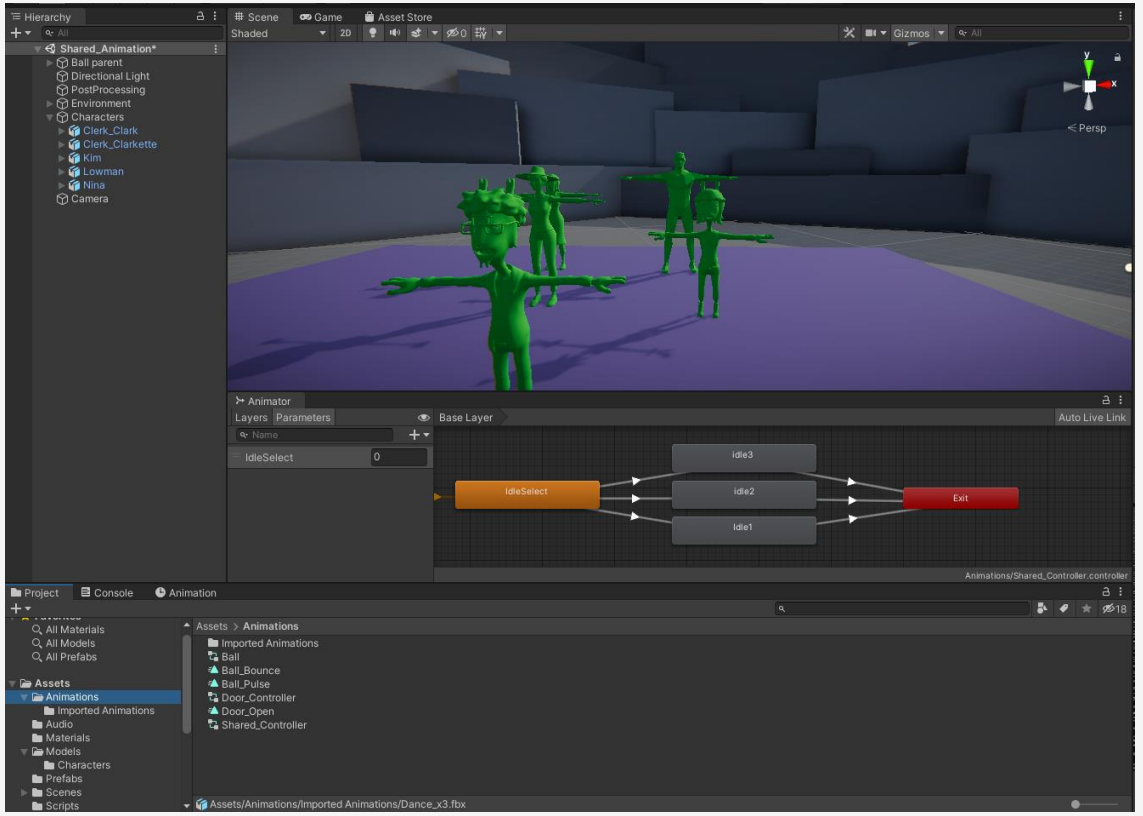
  8. Enable **Loop Time**.

  9. Enable the **Bake into Pose** option in Root Transform Rotation, Root Transform Position (Y), and Root Transform Position (XZ).

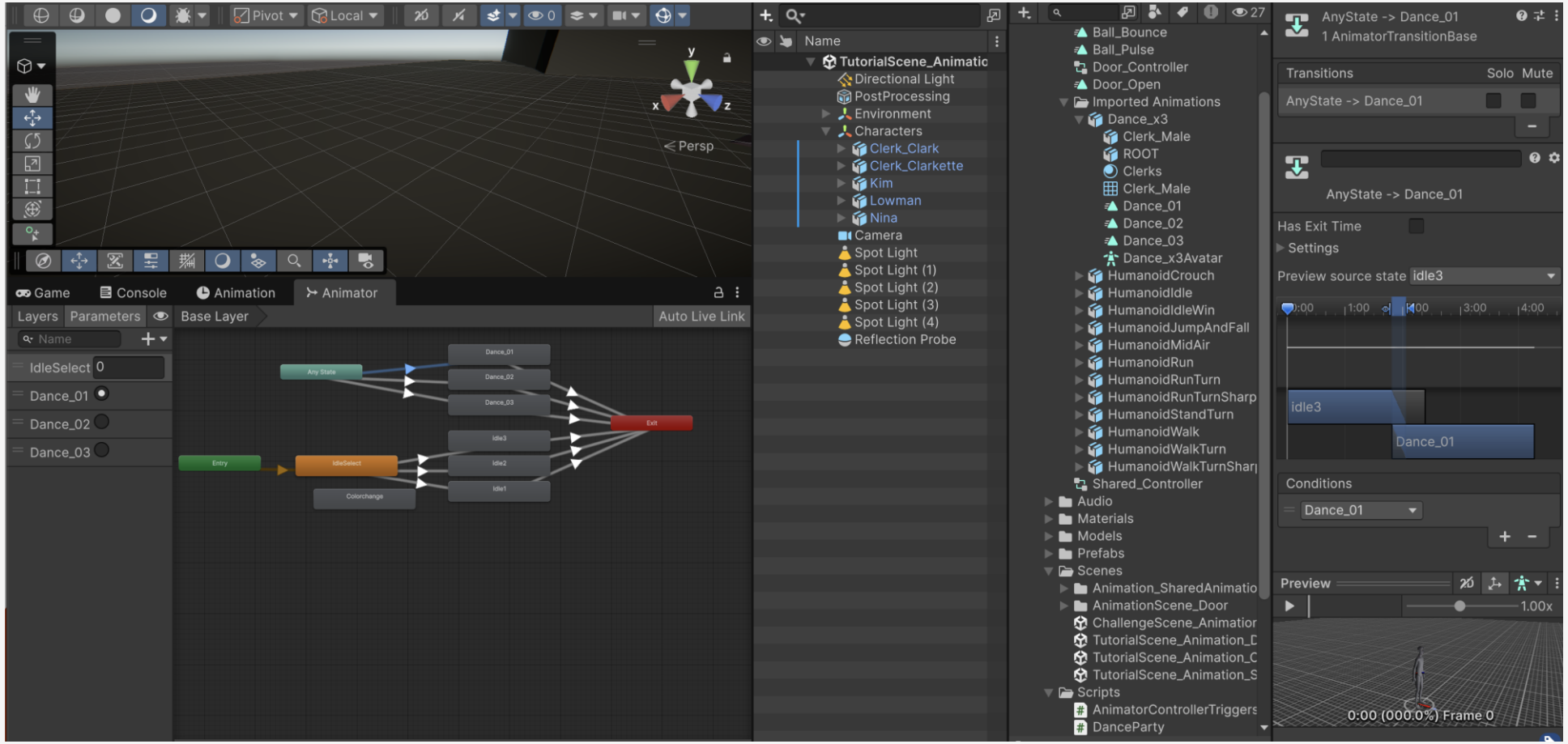
  10. Scroll down to the very bottom of the inspector and select Apply to save all of your import changes.

Add the new clips to the animation controller

- The new clips are now fully configured and can be added to the already existing Animator Controller.
- **1.** In the animations folder, locate the Shared_Controller Animation Controller and double-click to open it in the Animator.
- **2.** Dock the Animator directly below the Scene view so you have access to the Project window and the Animator at the same time.
- **3.** Locate the Dance_x3 animation in the Imported Animations folder and expand it to reveal all three new Animation Clips.
- Attached to each of the characters in the scene is a script called DanceParty. If you'd like to review it in Visual Studio, double-click it to open it. The script's functionality is simple; when the application runs, the script accesses each character's animator and will begin looking for keyboard input from the user. If the user presses Q, W, or E, then the script will call a Trigger Parameter named Dance_01, Dance_02, or Dance_03 respectively.



Create 3 parameters in the Animator and connect them to the new Animation Clips



DanceParty

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;


public class DanceParty : MonoBehaviour
{
private Animator anim;


// Start is called before the first frame update
void Start()
{
anim = GetComponent<Animator>();
}
// Update is called once per frame
void Update()
{
if (Input.GetKeyDown(KeyCode.Q))
{
anim.SetTrigger("Dance_01");
}
if (Input.GetKeyDown(KeyCode.W))
{
anim.SetTrigger("Dance_02");
}

if (Input.GetKeyDown(KeyCode.E))
{
anim.SetTrigger("Dance_03");
}

if (Input.GetKeyDown(KeyCode.R))
{
anim.SetTrigger("Dance_04");
}

if (Input.GetKeyDown(KeyCode.T))
{
anim.SetTrigger("Dance_05");
}

if (Input.GetKeyDown(KeyCode.Y))
{
anim.SetTrigger("Dance_06");
}
}
}
```